

Documentacao da API

Simulador Estrategico de Loja | Squad 16

Base URL: <http://localhost:3000/api>

Versao 1.0 | Abril 2026

GET

POST

PUT

PATCH

DELETE

Esta API RESTful e o backbone do Simulador Estrategico de Loja, uma plataforma de treinamento em varejo desenvolvida para a Cencosud. Ela expoe recursos para autenticacao, gerenciamento de usuarios/squads/produtos, controle de estoque, execucao de rodadas de simulacao e calculo de resultados financeiros (DRE).

Toda requisicao (exceto /auth/login) exige o header `Authorization: Bearer <token>` obtido no login. O sistema implementa RBAC com tres papeis: GAME_MASTER, PLAYER e OBSERVER.

1. Autenticacao

POST /auth/login

Acesso: Publico

Descricao: Autentica o usuario e retorna um JWT valido por 8 horas.

Request:

```
// Request Body
{
  "email": "admin@simulador.com",
  "password": "admin123"
}
```

Response:

```
// 200 OK
{
  "token": "<jwt>",
  "user": {
    "id": "cuid...",
    "name": "Administrador",
    "role": "GAME_MASTER",
    "squadId": null
  }
}

// 400 Bad Request - validacao
```

```
// 401 Unauthorized - credenciais invalidas
```

O payload do JWT contem: { userId, role, squadId }. Inclua o token em todas as chamadas subsequentes via header Authorization: Bearer <token>.

2. Usuarios /users

Todos os endpoints exigem role GAME_MASTER.

GET	/users	GAME_MASTER	Lista todos os usuarios
POST	/users	GAME_MASTER	Cria novo usuario
PUT	/users/:id	GAME_MASTER	Atualiza dados do usuario
DELETE	/users/:id	GAME_MASTER	Remove usuario

Exemplo - criacao de usuario:

```
// POST /users - Body
{
  "name": "Joao Silva",
  "email": "joao@simulador.com",
  "password": "senha123",
  "role": "PLAYER"      // GAME_MASTER | PLAYER | OBSERVER
}
```

3. Produtos /products

GET e acessivel a GAME_MASTER e PLAYER; demais operacoes apenas GAME_MASTER.

GET	/products	GM+PLAYER	Lista todos os produtos
POST	/products	GAME_MASTER	Cria produto
PUT	/products/:id	GAME_MASTER	Atualiza produto
DELETE	/products/:id	GAME_MASTER	Remove produto

Exemplo - cadastro de produto:

```
// POST /products - Body
{
  "name": "Arroz",
  "purchasePrice": 3.50      // preco de custo (number)
}
```

4. Squads /squads

Todos os endpoints exigem role GAME_MASTER.

GET	/squads	GAME_MASTER	Lista todos os squads
-----	---------	-------------	-----------------------

POST	/squads	GAME_MASTER	Cria squad
PUT	/squads/:id	GAME_MASTER	Atualiza squad
DELETE	/squads/:id	GAME_MASTER	Remove squad
POST	/squads/:id/users	GAME_MASTER	Adiciona usuario ao squad
DELETE	/squads/:id/users/:userId	GAME_MASTER	Remove usuario do squad

5. Lojas /stores

GET	/stores	GAME_MASTER	Lista todas as lojas
POST	/stores	PLAYER	Cria loja para o proprio squad
GET	/stores/my	PLAYER	Retorna loja do squad autenticado

Exemplo - criacao de loja:

```
// POST /stores - Body
{
  "name": "Loja Alpha"
}
```

6. Inventario /stores/:storeId/inventory

GET	/stores/:storeId/inventory	GM+PLAYER	Lista estoque da loja
PUT	/stores/:storeId/inventory/:productId	GAME_MASTER	Atualiza qtd. de produto
POST	/stores/:storeId/inventory/restock	GAME_MASTER	Reabastece produtos em lote

Exemplos - inventario:

```
// PUT /stores/:storeId/inventory/:productId - Body
{ "quantity": 150 }

// POST /stores/:storeId/inventory/restock - Body
{
  "items": [
    { "productId": "cuid...", "quantity": 100 },
    { "productId": "cuid...", "quantity": 80 }
  ]
}
```

7. Rodadas /rounds

GET	/rounds	GM+PLAYER	Lista todas as rodadas
GET	/rounds/:id	GM+PLAYER	Detalhes de uma rodada

POST	/rounds	GAME_MASTER	Cria rodada (status OPEN)
PATCH	/rounds/:id/close	GAME_MASTER	Processa e encerra rodada
POST	/rounds/:id/config	PLAYER	Submete estrategia do squad
GET	/rounds/:id/results	GM+PLAYER	Retorna resultados financeiros

Criacao de rodada:

```
// POST /rounds - Body
{
  "name": "Rodada 1",
  "startDate": "2026-04-14T00:00:00Z",
  "endDate": "2026-04-21T00:00:00Z"
}
```

Submissao de estrategia (config):

```
// POST /rounds/:id/config - Body (PLAYER)
{
  "fixedExpenses": 5000,
  "variableExpenses": 2000,
  "items": [
    { "productId": "cuid...", "salePrice": 6.50, "salesVolume": 80 },
    { "productId": "cuid...", "salePrice": 8.00, "salesVolume": 60 }
  ]
}

// 201 Created
{
  "roundConfigId": "cuid...",
  "storeId": "cuid...",
  "roundId": "cuid...",
  "fixedExpenses": 5000,
  "variableExpenses": 2000,
  "submittedAt": "2026-04-14T10:30:00Z",
  "items": [ ... ]
}
```

Resultados financeiros:

```
// GET /rounds/:id/results - Resposta PLAYER
{
  "grossRevenue": 2600.00,
  "costs": 840.00,
  "grossProfit": 1760.00,
  "expenses": 7000.00,
  "netProfit": -5240.00,
  "netMargin": -201.5,
  "store": { "id": "...", "name": "Loja Alpha", "squadId": "..."},
  "roundConfig": { ... }
}

// Resposta GAME_MASTER: array com resultados de todas as lojas
```

8. Simulacao /simulation

POST	/simulation/preview	PLAYER	Calcula DRE sem persistir
GET	/simulation/ranking	GM+PLAYER	Ranking da rodada (?roundId=...)

Preview de DRE:

```
// POST /simulation/preview - Body (mesmo schema de /rounds/:id/config)
{
  "fixedExpenses": 5000,
  "variableExpenses": 2000,
  "items": [
    { "productId": "cuid...", "salePrice": 6.50, "salesVolume": 80 }
  ]
}

// 200 OK
{
  "preview": true,
  "dre": {
    "grossRevenue": 520.00,
    "costs": 280.00,
    "grossProfit": 240.00,
    "expenses": 7000.00,
    "netProfit": -6760.00,
    "netMargin": -1300.0
  },
  "feedbacks": [
    "Prejuizo detectado. Revise seus precos ou reduza despesas.",
    "Margem liquida negativa. Considere aumentar o volume de vendas."
  ]
}
```

Ranking:

```
// GET /simulation/ranking?roundId=<cuid>
[
  { "rank": 1, "storeName": "Loja Beta", "netProfit": 3200.00, "netMargin": 28.5 },
  { "rank": 2, "storeName": "Loja Alpha", "netProfit": -5240.00, "netMargin": -201.5 }
]
```

9. Regras de Negocio - Calculo DRE

effectiveVolume	min(salesVolume, availableStock) - limitado ao estoque disponivel
grossRevenue	Soma de (salePrice x effectiveVolume)
costs	Soma de (purchasePrice x effectiveVolume)
grossProfit	grossRevenue - costs
expenses	fixedExpenses + variableExpenses

netProfit

grossProfit - expenses

$$\text{netMargin } (\%)$$

$$(\text{netProfit} / \text{grossRevenue}) \times 100$$

Feedbacks automaticos retornados em /simulation/preview:

- Prejuizo detectado - quando $\text{netProfit} < 0$
- Margem liquida abaixo de 10% - sinal de alerta
- Estoque insuficiente - quando $\text{salesVolume} > \text{availableStock}$ para algum produto

10. Fluxo do Jogo

1	GM cria rodada	POST /rounds	-> status: OPEN
2	Players montam loja	POST /stores	(uma vez por squad)
3	GM abastece estoque	POST /stores/:id/inventory/restock	
4	Players simulam DRE	POST /simulation/preview	(sem persistir)
5	Players submetem config	POST /rounds/:id/config	(uma vez por rodada)
6	GM encerra rodada	PATCH /rounds/:id/close	-> status: CLOSED
7	Resultados disponiveis	GET /rounds/:id/results + GET /simulation/ranking	

11. Respostas de Erro Padrao

Codigo	Status	Situacao
400	Bad Request	Dados de entrada invalidos (Zod)
401	Unauthorized	Token ausente, invalido ou expirado
403	Forbidden	Role sem permissao para o recurso
404	Not Found	Recurso nao encontrado
409	Conflict	Conflito (ex: config ja submetida, rodada nao OPEN)
500	Server Error	Erro interno - verificar logs do Express

Formato de erro:

```
// Formato padrao de erro
{ "message": "Descricao do erro" }

// Erros de validacao (Zod)
{
  "errors": {
    "email": ["Email invalido"],
    "password": ["Senha deve ter no minimo 6 caracteres"]
  }
}
```

12. Dados de Seed (Ambiente de Desenvolvimento)

```
// Usuarios
admin@simulador.com / admin123 -> GAME_MASTER
```



```
alpha@simulador.com / player123 -> PLAYER (Squad Alpha)
beta@simulador.com / player123 -> PLAYER (Squad Beta)

// Produtos
Arroz | Feijao | Macarrao | Leite | Oleo

// Squads & Lojas
Squad Alpha -> Loja Alpha (100 unid./produto)
Squad Beta -> Loja Beta (100 unid./produto)
```